

- `image = models.ImageField()`
- `def __str__(self):`
- `return self.name`
- Now our Image model:
- `class Image(models.Model):`
- `name = models.CharField(max_length=32)`
- `slug = models.SlugField()`
- `alt_text = models.CharField(max_length=255)`
- `image = models.ImageField()`
- `def __str__(self):`
- `return self.name`

You might be wondering, why do we have a separate model for image that itself has a field that actually contains the image. Why not just directly use the image field wherever needed?

There are two reasons for us to do this, firstly this allows us to store additional data about the image such as its name and some alternate text; secondly we have to have a fixed number of fields in a model, so we cannot have a gallery model that allows an unlimited number of Images unless images are stored as a model rather than as a field.

Now our model for galleries:

- `class Gallery(models.Model):`
- `name = models.CharField(max_length=32)`
- `slug = models.SlugField()`
- `images = models.ManyToManyField(Image)`
- `description = models.TextField()`
- `layout = models.CharField(`
- `max_length=5,`
- `choices=(`
- `(‘LINK’, ‘Linked’),`
- `(‘HORI’, ‘Horizontal’),`
- `(‘VERT’, ‘Vertical’))`
- `def __str__(self):`
- `return self.name`

Here you should see some things worth talking about. Firstly, the layout field that has a choices parameter which is a tuple of tuples. This is Django’s